

Tensor Omics: Data Model Definitions

Asis Hallab, “el Bobito”

May 8, 2025

1 Data Models

All data in Tensor Omics (TOX) must be stored continuously in memory in basic Fortran types. All data must be serializable and deserializable from binary files (`.txdata` format). Those files consist of a header section informing about the contained basic Fortran structures (scalars and arrays), their sizes, and the offset of where to start to read them.

Tensor Omics will require meta-data to be readable and writable from data tables, e.g. `CSV` format. To represent this efficiently we will use the below `DataTable` definitions.

1.1 DataTable

1.1.1 Overview

The **F42 DataTable** implements the **F42 DataStructure** standard. It provides a compact, efficient, and portable representation of structured tabular data (e.g., parsed from `CSV` files). It is optimized for serialization, patching, and memory-constrained environments.

1.1.2 Public Methods

- `read_from_file(file, sep_char, column_types, max_char_length, n_rows, column_names)`
- `calculate_memory_requirements()`
- `serialize()`
- `deserialize()`
- `log()`
- `update()` (Detailed definition later)
- `save_patch()` (Detailed definition later)

1.1.3 Internal Structure

The `DataTable` is composed of the following internal components:

- **Type-Mapping Array:** a $2 \times k$ integer array.
 - Row 1: Stores the column type for each input column.
 - Row 2: Stores the index of the corresponding column inside the appropriate data container (see below).
- **Data Containers:**
 - `int_array`: A $n_rows \times i$ array for integer columns.
 - `real_array`: A $n_rows \times r$ array for real (floating-point) columns.

- **char_array**: A $n_rows \times c$ array of fixed-length character sequences (length = max_char_length) for string columns.

where $i + r + c = k$, and each value $i, r, c \geq 0$.

- **Column Names:**

- A $k \times 256$ character array.
- Each entry holds the name of a column, truncated or padded to a maximum length of 256 characters.

1.1.4 Detailed Method Definitions

read_from_file

Method signature: `read_from_file(file, sep_char, column_types, max_char_length, n_rows, column_names)`

Reads tabular data from a file and populates the internal structure.

- **file**: Path or handle of the input file.
- **sep_char**: Character used to separate fields (e.g., ‘,’ or ‘;’).
- **column_types**: Integer array of length k specifying the type of each input column:
 - 1 = Integer
 - 2 = Real (floating point)
 - 3 = Character (string)
- **max_char_length**: Maximum allowed character length for each string field (in char columns).
- **n_rows**: Expected number of data rows (excluding headers).
- **column_names** (optional): Array of column names. If omitted, defaults to generic names (“col1”, “col2”, ...).

calculate_memory_requirements

Method signature: `calculate_memory_requirements(n_rows, col_types)`

Estimates and returns the total memory (in bytes) required for the DataTable, including:

- Size of Type-Mapping Array: $2 \times k \times \text{sizeof}(\text{int})$
- Size of Data Containers:
 - $n_rows \times i \times \text{sizeof}(\text{int})$
 - $n_rows \times r \times \text{sizeof}(\text{real})$
 - $n_rows \times c \times \text{max_char_length} \times \text{sizeof}(\text{char})$
- Size of Column Names Array: $k \times 256 \times \text{sizeof}(\text{char})$

serialize()

Serializes the DataTable into a binary format consisting of:

1. Header block (generated by **generate_header**).
2. Raw memory blocks for:
 - Type-Mapping Array
 - `int_array`
 - `real_array`
 - `char_array`
 - `column_names`

All blocks are concatenated sequentially without gaps. Serialization follows little-endian ordering unless otherwise specified.

generate_header()

Creates a header structure containing metadata such as:

- Magic number (identifies the file type, e.g., 'F42DTBL')
- Version number
- Number of rows (n_rows)
- Number of columns (k)
- Maximum character length (max_char_length)
- Column counts per type (i, r, c)

deserialize()

Inverse operation of **serialize()**:

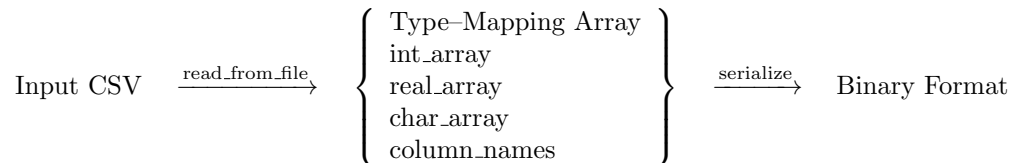
- Reads and verifies the header.
- Reconstructs the Type-Mapping Array and Data Containers from the binary block.
- Ensures compatibility of all dimensions and memory alignment.

1.1.5 Methods to be Specified Later

The following methods will be specified in detail in a future document:

- `log()`
- `update()`
- `save_patch()`

1.1.6 Summary Diagram



2 F42-Compliant Nested HashMap System

2.1 Overview

The F42 HashMap system provides an efficient, statically typed representation of arbitrarily nested key-value maps using flat, preallocated memory bands. Each scalar type is stored in a dedicated array (Turing Band). Keys and properties are indexed via integer references, enabling fast lookup, nesting, and serialization.

2.2 Memory Bands and Data Layout

- **Value Bands** (type-specific storage):

- `int_vals(n_int)`
- `real_vals(n_real)`
- `char_vals(n_char, len_char)`

- **Key Index Table:**

`keys(3, n_maps)` contains for each HashMap entry:

- `keys(1,i)`: Index of the key string in `char_vals`
- `keys(2,i)`: Start index in `properties` table
- `keys(3,i)`: Stop index in `properties` table

- **Properties Table:**

`properties(4, n_properties)`

Each column corresponds to a field in a map:

- `properties(1,j)`: Field name (index in `char_vals`)
- `properties(2,j)`: Type code (1 = int, 2 = char, 3 = real, 4 = nested map)
- `properties(3,j)`: Start index in the corresponding value band
- `properties(4,j)`: Stop index (for vector values or nested maps)

2.3 Access Function Pseudocode

Lookup Key in HashMap

```
function get_map_range(keys, key_id):
  do i = 1, size(keys, 2)
    if keys(1, i) == key_id then
      return keys(2, i), keys(3, i) ! Return property start/stop indices
    end if
  end do
  return -1, -1 ! Not found
```

Lookup Property in Map

```
function get_property(properties, i_start, i_stop, field_id):
  do j = i_start, i_stop
    if properties(1, j) == field_id then
      type_id = properties(2, j)
      val_start = properties(3, j)
      val_stop = properties(4, j)
      return type_id, val_start, val_stop
    end if
  end do
  return -1, -1, -1 ! Not found
```

Retrieve Value Once type and start/stop indices are known:

```
if type_id == 1:
    return int_vals(val_start:val_stop)
if type_id == 2:
    return char_vals(val_start:val_stop, :)
...
if type_id == 4:
    return nested map keys(:, val_start:val_stop)
```

2.4 Serialization and Nested Maps

- Entire structure can be written as binary blocks:
 - keys, properties, value bands
- Nested maps are supported via:
 - type_id = 4
 - val_start, val_stop refer to rows in keys()
- Recursive traversal follows key -> property -> nested map pattern.

2.5 Advantages

- Fully F42-compliant: flat arrays, preallocated memory, no dynamic allocation.
- Serialization-ready.
- Recursive, nested structure without pointers.
- Clean and performant traversal and update patterns.

2.6 Test Cases for F42 Nested HashMaps

Test 1: Simple Flat Key-Value Retrieval

```
! Setup:
! char_vals: ["name", "age"]
! int_vals: [42]
! keys(1,1) = index("patient1") in char_vals
! keys(2,1:3,1) = (1,2)          ! property range

! properties(:,1) = [index("name"), 2, 1, 1]
! properties(:,2) = [index("age"), 1, 1, 1]

! Test retrieval of age
call get_map_range(keys, index("patient1")) => (1, 2)
call get_property(properties, 1, 2, index("age")) => (1, 1, 1)
int_vals(1) = 42 ! -> OK
```

Test 2: Nested Map Traversal

```
! Setup:
! keys(1,1) = index("patient1")
! properties(:,2) = [index("clinical_data"), 4, 2, 2]
! keys(:,2) = nested map root: index("diagnosis"), range (3,3)
! properties(:,3) = [index("diagnosis"), 2, 3, 3]
```

```

! char_vals(3,:) = "healthy"

! Access:
call get_map_range(keys, index("patient1")) => (1, 2)
call get_property(properties, 1, 2, index("clinical_data")) => (4, 2, 2)
call get_map_range(keys, 2) => (3, 3)
call get_property(properties, 3, 3, index("diagnosis")) => (2, 3, 3)
char_vals(3,:) = "healthy" ! -> OK

```

Test 3: Missing Key or Property Handling

```

! Try to get nonexistent key "ghost"
call get_map_range(keys, index("ghost")) => (-1, -1) => OK

! Try to get nonexistent field "weight" in patient1
call get_map_range(keys, index("patient1")) => (1,2)
call get_property(properties, 1, 2, index("weight")) => (-1, -1, -1) => OK

```

Test 4: Multiple Fields of Same Type

```

! Add "height" and "weight" of type real
! properties(:,4) = [index("height"), 3, 1, 1]
! properties(:,5) = [index("weight"), 3, 2, 2]
! real_vals(1) = 1.75
! real_vals(2) = 70.0

! Test:
call get_property(properties, 1, 5, index("weight")) => (3, 2, 2)
real_vals(2) = 70.0 => OK

```

Test 5: Field as Vector Value

```

! properties(:,6) = [index("expression"), 3, 3, 5]
! real_vals(3:5) = [1.2, 0.9, 3.1]

! Retrieve vector value
call get_property(properties, 1, 6, index("expression")) => (3, 3, 5)
real_vals(3:5) = [1.2, 0.9, 3.1] => OK

```

Validation Summary Each of these tests ensures:

- Correct indexing of key and property arrays
- Accurate lookup and retrieval in Turing Bands
- Support for scalar and vector fields
- Support for nested maps via integer nesting
- Clean handling of missing keys or fields

Further tests may involve performance profiling, binary serialization validation, and nested structure export/reconstruction checks.

3 F42 Hashmap Memory and Update Semantics (Extended)

3.1 Compaction and Memory Hygiene

To maintain efficient memory use and avoid fragmentation due to property overwrites or deletions, we define an explicit compaction mechanism for all Turing bands and relevant 2D structures such as the ‘keys’ and ‘properties’ tables.

3.1.1 Compaction Strategy

- For each band (e.g., `int_vals`, `char_vals`, `real_vals`), maintain an accompanying integer array `deleted_indices(:)` containing the indices of entries marked as obsolete.
- During compaction:
 1. Allocate a new compacted array of minimal size.
 2. Copy only non-obsolete entries.
 3. Record an index remapping from old to new positions.
 4. Traverse the `properties` array and remap all band references accordingly.
 5. Replace the old band with the new one.
- This strategy supports all scalar types and nested `HashMap` structures uniformly.
- **Do not embed deletion markers in the bands themselves**, as that strategy fails for fixed-length character arrays and would violate F42 separation-of-concerns principles.
- Logical deletion arrays also permit filtered read-only access (e.g., using `WHERE (.NOT. is_deleted)`) without triggering compaction.

3.1.2 HashMap Lifecycle Pseudocode (extended)

```
subroutine compact_band(old_band, deleted_indices, new_band, index_map)
  integer, intent(in) :: old_band(:), deleted_indices(:)
  integer, intent(out) :: new_band(:), index_map(size(old_band))

  ! Step 1: mark deletions
  logical :: keep(size(old_band))
  keep = .true.
  keep(deleted_indices) = .false.

  ! Step 2: create new_band and remap indices
  new_band = pack(old_band, keep)
  new_pos = 1
  do i = 1, size(old_band)
    if (keep(i)) then
      index_map(i) = new_pos
      new_pos = new_pos + 1
    else
      index_map(i) = -1
    end if
  end do

  ! Step 3: update property lookup intervals using index_map
end subroutine

type(HashMapCollection) :: mapset
```

```

call compact_band(mapset%int_vals, mapset%int_deleted, compacted, remap)

doit: do i = 1, mapset%num_properties
  if (mapset%properties(2, i) == 1) then
    mapset%properties(3:4, i) = remap(mapset%properties(3:4, i))
  end if
end do doit

```

This strategy provides robust, scalable memory hygiene compatible with both Fortran memory models and F42 minimalistic reproducibility.

3.2 F42-Compliant Graph Representation Using Flat Arrays

3.2.1 Design Overview

The F42 graph structure is implemented using a pair of flat 2D arrays to represent connectivity and node/edge labeling. This design preserves strict memory control, serialization efficiency, and WebAssembly compatibility.

- **Adjacency Matrix** `adj(n, m)`
Each entry at `adj(i, j)` holds the label index (column in the label matrix) of the node to which node `i` is connected via edge `j`.
- **Label Matrix** `labels(2, k)`
 - Row 1: Label type.
1 = DataTable row
2 = HashMap reference
 - Row 2: Label index (row or key index).

The first `n` columns in `labels` refer to node labels; additional columns represent edge labels.

3.2.2 Pseudocode: Graph Construction

```

procedure initialize_graph(adj, labels, n, m)
  ! Allocate adj(n, m)
  ! Allocate labels(2, n + m) ! n node labels, m edge labels
  set all entries of adj to -1
  set all labels to -1
end procedure

procedure add_node_label(labels, node_index, type_code, index)
  labels(1, node_index) = type_code
  labels(2, node_index) = index
end procedure

procedure add_edge(adj, labels, from_node, edge_slot, to_node_label_index)
  adj(from_node, edge_slot) = to_node_label_index
end procedure

procedure add_edge_label(labels, edge_label_index, type_code, index)
  labels(1, edge_label_index) = type_code
  labels(2, edge_label_index) = index
end procedure

```

3.2.3 Key Properties and Advantages

- Fully flat structure; ideal for streaming, serialization, and WASM.

- Separation of node/edge data from topology.
- Labels act as foreign keys into canonical data structures like `DataTable` or `HashMap`.
- Easy to extend for directed/undirected graphs or hypergraphs by modifying adjacency encoding.
- Compatible with graph algorithms via simple indexing and vector traversal.

3.2.4 Test Case: Simple Directed Graph

Example: A directed graph with 3 nodes (A, B, C) and 2 directed edges: $A \rightarrow B$ and $B \rightarrow C$.

```
initialize_graph(adj, labels, 3, 2)

add_node_label(labels, 1, 1, 5)  ! Node A points to row 5 of DataTable
add_node_label(labels, 2, 1, 8)  ! Node B points to row 8
add_node_label(labels, 3, 1, 9)  ! Node C points to row 9

add_edge(adj, labels, 1, 1, 2)    ! A → B
add_edge(adj, labels, 2, 1, 3)    ! B → C
```

This structure can now be queried, visualized, or extended without reallocation or symbolic interpretation.